



Praktikum Mikrocontroller PO2019

Versuch 1 Einführung in das M5Stack-Core2-Board

Name, Vorname	Matrikelnummer
Versuchstag	Testat

1. Versuchsbeschreibung

In diesem Versuch sollen Sie das im Labor installierte Python-Board kennenlernen und dort erste einfache Programme testen.

1.1 Hardware

Bei dem hier verwendeten Board handelt es sich um ein M5Stack-Core2-Board mit einem ESP32-Mikrocontroller.

Die Original M5Stack-Firmware ist gegen ein Standard-Micropython mit einigen zusätzlich benötigten Bibliotheken ausgetauscht worden.

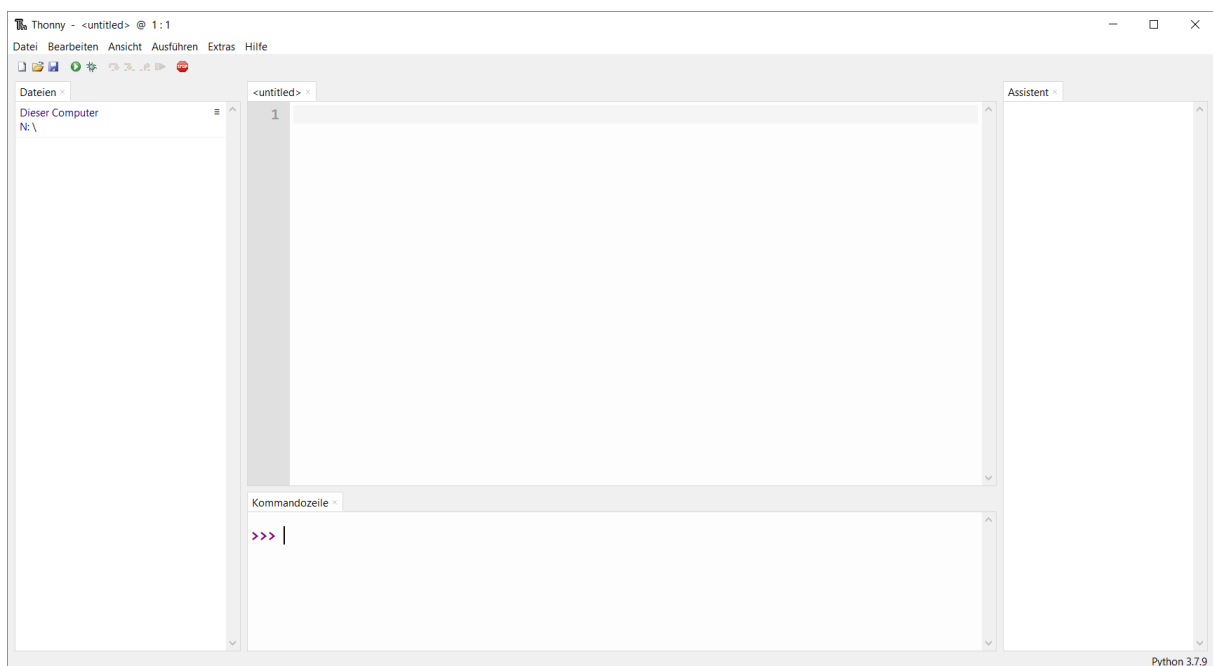


- ESP32-D0WDQ6-V3-Controller
- 16MB Flash und 8MB PSRAM
- USB-C-Anschluss mit CP2104-Controller
- 2" 320x240 Farb-LCD mit ILI9342C-Controller

- FT6336U-Touchscreen
- 6-Achsen IMU MPU6886
- BM8563-RTC
- AXP192 Power Management Unit
- NS4168-Verstärker mit Lautsprecher
- SPM1423-Mikrofon
- Vibrationsmotor
- SD-Karten-Slot

1.2 Entwicklungsumgebung

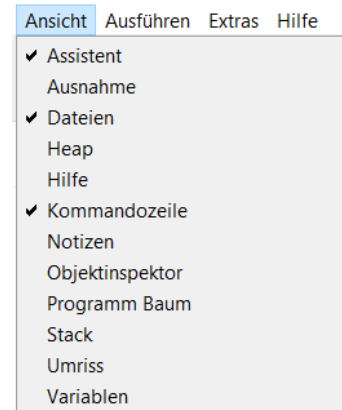
Als Entwicklungsumgebung verwenden wir in diesem Versuch die frei verfügbare Software **Thonny**.



Kontrollieren Sie nach dem Start von Thonny zunächst die Einstellungen im Menü **Ansicht**. Dort sollten die Menüpunkte

- Assistent
- Dateien
- Kommandozeile

ausgewählt sein.



1.3 Verbindung mit dem Mikrocontroller

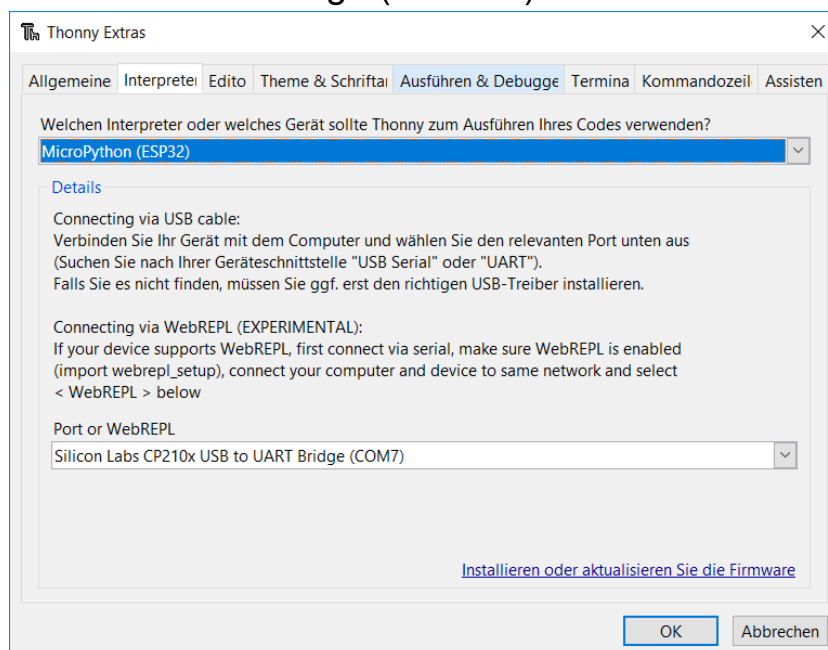
Die USB-C Schnittstelle des M5Stack-Core2-Controllers ist mit einem USB-Kabel mit dem PC verbunden.

Um aus der Thonny-IDE den Mikrocontroller anzusprechen müssen Sie zunächst im Menü **Ausführen** den von Thonny verwendeten Python-Interpreter auswählen.

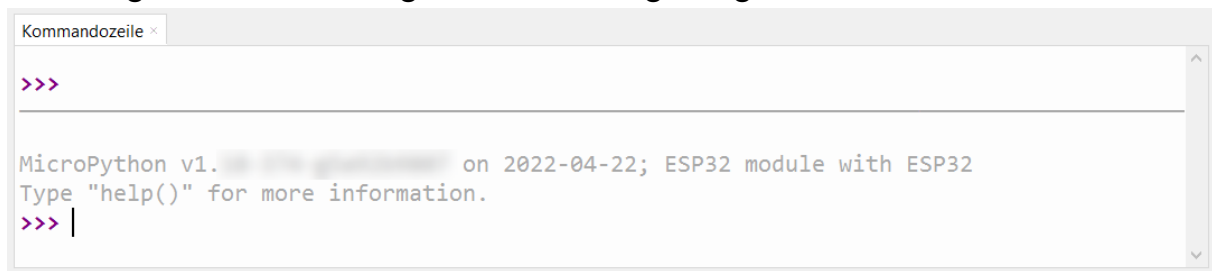
Wählen Sie in dem angezeigten Dialog als Gerät / Interpreter **MicroPython (ESP32)**.

Die zur Verbindung mit dem Mikrocontroller benötigte serielle Schnittstelle kann danach im unteren Bereich des Dialogs ausgewählt werden.

Die richtige Schnittstelle erkennen Sie an dem Chip der angezeigten Silicon Labs USB-Bridge (CP210x).



Wenn Sie den Dialog mit dem OK-Button schließen sollte unten im Kommandozeilen-Bereich von Thonny die MicroPython-Startmeldung mit der Eingabeaufforderung der REPL angezeigt werden.



```
Kommandozeile x
>>>

MicroPython v1.20.0 on 2022-04-22; ESP32 module with ESP32
Type "help()" for more information.
>>> |
```

Welche MicroPython-Version wird hier angezeigt?

Innerhalb von Python können Sie nach dem Import der **sys**-Bibliothek mit **sys.version** die Python-Version anzeigen auf der das verwendete MicroPython basiert.

Welche Version wird hier angezeigt?

Um die Version der angepassten Labor-Firmware herauszufinden können Sie aus dem REPL-Prompt mit der Anweisung **import sysinfo** einen Statusbildschirm aufrufen, der gleichzeitig als Test für das Display dient.

Welche Version wird hier angezeigt?

2. Versuchsdurchführung

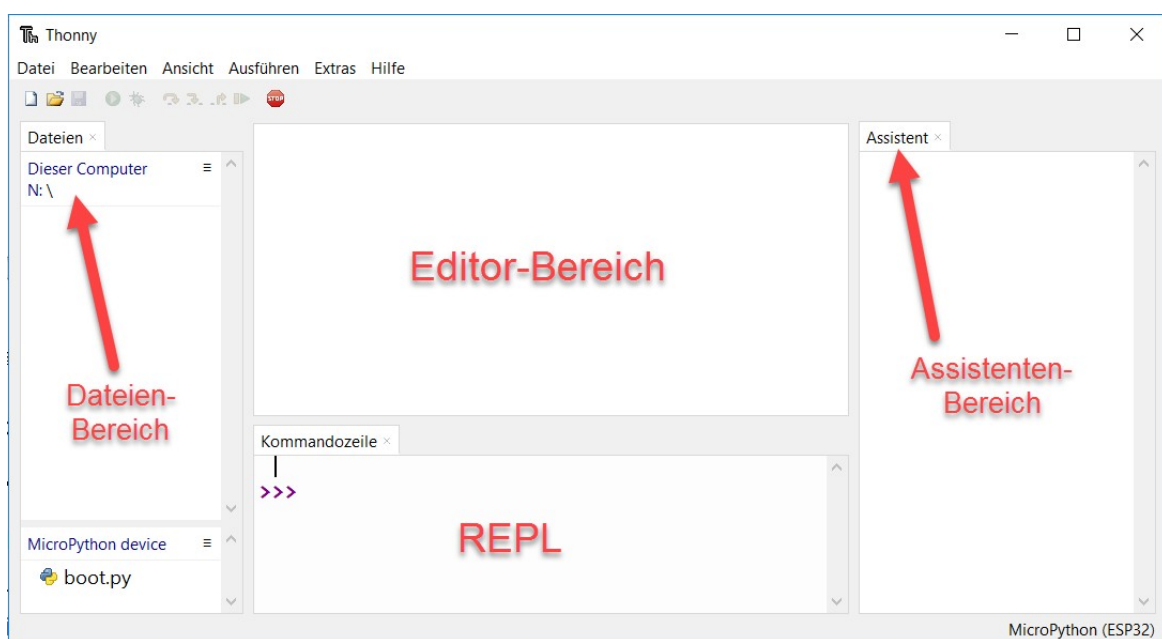
2.1 Hallo Welt

Um die Handhabung der Entwicklungsumgebung kennenzulernen sollen Sie zunächst ein einfaches Programm mit Textausgabe schreiben. Die Entwicklungsumgebung ist in drei horizontal nebeneinander angeordnete Bereiche unterteilt. Der linke Bereich **Dateien** zeigt Ihnen oben Dateien im lokalen Dateisystem des PC und unten Dateien im Dateisystem des Mikrocontrollers an.

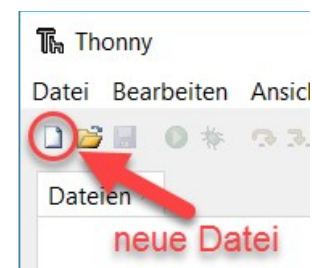
Der Rechte Bereich **Assistent** zeigt Ihnen Hilfestellungen zu Ihrem Programm an und wird momentan nicht benötigt.

Der mittlere Bereich hat keinen Titel und enthält in einem oder mehreren Reitern die zu bearbeitenden Python-Quellcodes. Falls in diesem Bereich noch Reiter geöffnet sind schließen Sie zuerst diese Reiter.

(Schließen Sie NICHT die Bereiche **Dateien**, **Assistent** und **Kommandozeile**!)

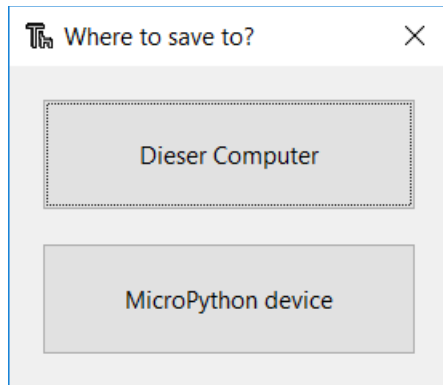


Öffnen Sie jetzt mit dem entsprechenden Button oder über das Datei-Menü eine neue Datei.



Der Reiter für die neu angelegte Datei enthält zunächst den Namen `<untitled>`.

Speichern Sie die neue Datei über den entsprechenden Menüpunkt **Datei/Speichern**.



Bei jedem Speichern fragt die Entwicklungsumgebung zunächst, ob die Datei auf dem PC oder auf dem Mikrocontroller gespeichert werden soll.

Dauerhaft benötigte Dateien können Sie so auf dem Mikrocontroller speichern. Hier bietet sich die Speicherung auf dem Laufwerk **n:** an.

Auf dem Controller sollte sich zum Beginn des Praktikums nur eine Datei mit dem Namen **boot.py** befinden.

Speichern Sie die neu angelegte Datei mit dem Namen **hallo.py**

Die Beschriftung des Reiters im Editor enthält anschließend den neuen Dateinamen. Dabei werden Dateien auf dem Mikrocontroller mit eckigen Klammern um den Namen gekennzeichnet.

Geben Sie jetzt im Editor-Bereich ein Programm ein, dass mit *print*-Anweisungen Ihren Namen und Ihre Matrikelnummer ausgibt.

Starten Sie das Programm mit einem Klick auf den grünen **RUN**-Button und beobachten Sie die Ausgabe im REPL-Bereich unten.

Achtung!

Löschen Sie bitte zum Ende des Praktikums alle von Ihnen auf dem Mikrocontroller gespeicherten Dateien. Nur die Datei **boot.py** gehört zum System und sollte nicht gelöscht werden.

Dateien die Sie auf dem Laufwerk n: angelegt haben bleiben erhalten.

Falls Sie Dateien auf dem Controller gespeichert haben sollten Sie diese Dateien zusätzlich auf dem **Laufwerk N:** des PC speichern.

2.2 Fibonacci-Zahlen

Die **Fibonacci**-Folge ist definiert für $n > 2$ als $f_n = f_{n-1} + f_{n-2}$ mit den Anfangswerten $f_1 = f_2 = 1$

- schreiben Sie ein Programm **fibonacci.py** in MicroPython, dass ZWEI Funktionen zur Ermittlung einer Fibonacci-Zahl enthält.
- Die Funktion **fibonacci** soll nicht rekursiv (Schleife!) programmiert sein.
- Die zweite Funktion **fibonacci_r** soll rekursiv programmiert sein.
- Die in der Bibliothek **time** enthaltene Funktionen **time.time_us()** und **time.time_ms()** liefert ihnen die seit dem Systemstart vergangene Zeit in **Mikrosekunden** oder **Millisekunden**.

Ermitteln Sie mit dieser Funktion die Laufzeit der beiden Fibonacci-Funktionen (ohne print!) für Übergabewerte 5, 10, 15 und 21.

Zahl	Fibonacci-Zahl	Laufzeit <i>fibonacci</i>	Laufzeit <i>fibonacci_r</i>
5			
10			
15			
21			

2.3 Ansteuerung des Displays

Der M5Stack-Core2-Controller ist mit einem 2“-Farb-LCD mit einer Auflösung von 320x240 Pixeln ausgestattet. Dieses Display wird mit einem Controller vom Typ **ILI9342C** angesteuert.

Neben der für das Display benötigten SPI-Schnittstelle ist zur Ansteuerung des Displays außerdem die Konfiguration der PMU (Power Management Unit) erforderlich.

Starten Sie ein neues Projekt **display.py**. In diesem Programm importieren Sie zunächst die für das Display, die PMU und die Schnittstellen benötigten Bibliotheken.

```
from machine import Pin, SPI
import ili9342c
import axp202c
```


Danach können Sie mit den eingebundenen Bibliotheken das Display zur Benutzung vorbereiten.

```
axp = axp202c.PMU(address=0x34) # PMU einbinden
axp.enablePower(axp202c.AXP192_LD02) # Display anschalten
axp.setDC3Voltage(3000) # Hintergrundbeleuchtung einstellen
spi = SPI(2,baudrate=6000000,sck=Pin(18),mosi=Pin(23)) # SPI init
tft = ili9342c.ILI9342C(spi,320,240,reset=Pin(33, Pin.OUT),
    cs=Pin(5, Pin.OUT),dc=Pin(15, Pin.OUT),rotation=0) # Display init
```

Nach diesen Zeilen ist das Display betriebsbereit und kann mit einer Reihe von Methoden des Display-Objekts angesteuert werden.

```
init()                # Display anschalten und löschen
fill(color)           # Display mit der angegebenen Farbe füllen
pixel(x,y,color)      # Pixel x/y auf die angegebene Farbe setzen
line(x0,y0,x1,y1,color) # Linie zwischen zwei Punkten
hline(x,y,length,color) # horizontale Linie (schnell)
vline(x,y,length,color) # vertikale Linie (schnell)
rect(x,y,width,height,color)      # Rechteck
fill_rect(x,y,width,height,color) # gefülltes Rechteck
text(font, s, x, y[,fg,bg]) # Text s mit dem Zeichensatz font ausgeben
rotation(rot)              # setzt die Display-Ausrichtung (0..3)
width()                   # gibt die Display-Breite zurück
height()                 # gibt die Display-Höhe zurück
color565(r,g,b)          # erzeugt rgb565-Farbwert aus RGB-Farben
```

Zur Angabe von Farben verwendet das Display das **rgb565**-Format. In diesem Format wird eine Farbe in zwei Bytes mit fünf Bits für Rot/Blau und sechs Bits für die Farbe grün in dem Format RRRRRGGGGGGBBBBB gespeichert. Zur Vereinfachung kennt die Display-Bibliothek acht vordefinierte Farben und eine Hilfsfunktion zum Erzeugen eines Farbwertes:

```
BLACK, BLUE, RED, GREEN, CYAN, MAGENTA, YELLOW, WHITE
```

Beispiel:

```
tft.init() # Initialisiert und löscht das Display
tft.fill(ili9342c.WHITE) # füllt das Bild weiß
tft.fill_rect(10,10,200,200,0x2f0) # farbiges Rechteck
tft.fill_rect(100,100,50,50,ili9342c.color565(255,128,0))
```

Wie viele verschiedene Farben kann das Display darstellen?

Zur Ausgabe von Text auf dem Display kann die **text**-Methode verwendet werden. Der dazu als Parameter benötigte Zeichensatz kann aus einer Reihe von vorhandenen Zeichensätzen vorher importiert werden.

```
vga1_16x16  
vga1_16x32  
vga1_8x16  
vga1_8x8  
vga1_bold_16x16  
vga1_bold_16x32  
vga2_16x16  
vga2_16x32  
vga2_8x16  
vga2_8x8  
vga2_bold_16x16  
vga2_bold_16x32
```

Beispiel:

```
import vga2_16x32 as font  
...  
tft.text(font, "Hochschule", 140, 15, ili9342c.RED, ili9342c.WHITE)
```

2.4 Hallo Welt 2

Erzeugen Sie ein neues Projekt **hallo2.py**. Geben Sie in diesem Programm Ihren Namen, Ihre Matrikelnummer und das aktuelle Datum auf dem Display aus. Verwenden Sie für jede der drei Angaben verschiedene Schriftarten und verschiedene Farben.

2.5 Farben

Erzeugen Sie ein neues Projekt **farben.py**. Geben Sie in diesem Programm auf einem weißen Hintergrund ein Raster von 3x3 Rechtecken in

verschiedenen Farben aus. Lassen Sie zwischen den Rechtecken einen kleinen Abstand (10 Pixel).

Wenn Sie die Bibliothek **random** einbinden können Sie mit der Funktion **random.randint(min,max)** einen Zufallswert im Bereich **min...max** erzeugen um damit eine zufällige Farbe anzuzeigen.

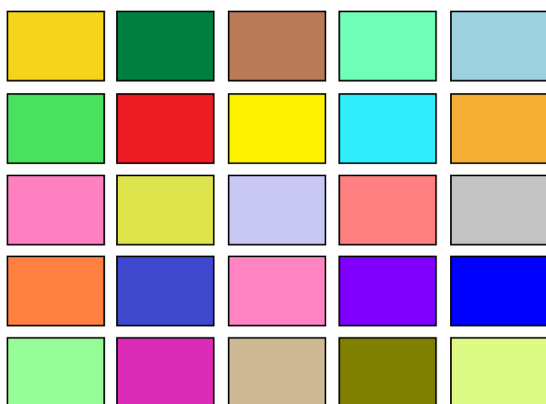


2.6 Farben 2

Versuchen Sie das Programm **farben.py** so zu ändern, dass ein Feld von **m** mal **n** Farbfeldern ausgegeben wird.

Die Werte für **m** und **n** können Sie dabei als Variablen im Programm festlegen.

Zeichnen Sie zusätzlich um jedes Farbfeld einen zwei Pixel breiten schwarzen Rand.



ACHTUNG!

Gehen Sie bei der Versuchsdurchführung in den angegebenen Schritten vor. Überspringen Sie keinen Schritt.

Dokumentieren/speichern Sie alle Schritte der Versuchsdurchführung so, dass Sie jeden Schritt vorzeigen können.

Löschen Sie bevor Sie Ihren Arbeitsplatz verlassen die von Ihnen auf dem Controller gespeicherten Dateien.